```
import time
import os
import random

WIDTH = 40
HEIGHT = 20
GENERATIONS = 200


def make_grid():
    grid = []

    for _ in range(HEIGHT):
        row = []

        for _ in range(WIDTH):
            # random initial state
            row.append(random.choice([0, 1]))

        grid.append(row)

    return grid


def count_neighbors(grid, x, y):
    alive = 0

    # iterate around current cell
    for dy in [-1, 0, 1]:
        for dx in [-1, 0, 1]:

            # skip current cell
            if dx == 0 and dy == 0:
                continue

            nx = (x + dx) % WIDTH
            ny = (y + dy) % HEIGHT

            alive += grid[ny][nx]

    return alive


def next_generation(grid):
```

```
    new_grid = []

    # main iteration over entire board
    for y in range(HEIGHT):
        row = []

        for x in range(WIDTH):
            neighbors = count_neighbors(grid, x, y)

            if grid[y][x] == 1:
                # Conway survival rules
                if neighbors == 2 or neighbors == 3:
                    row.append(1)
                else:
                    row.append(0)
            else:
                # Conway birth rule
                if neighbors == 3:
                    row.append(1)
                else:
                    row.append(0)

        new_grid.append(row)

    return new_grid


def draw(grid):
    os.system("clear")

    for row in grid:
        line = ""

        for cell in row:
            if cell:
                line += "#"
            else:
                line += " "

        print(line)


def main():
    board = make_grid()
```

```
    for generation in range(GENERATIONS):
        print(f"Generation: {generation}")

        draw(board)

        # iterative simulation update
        board = next_generation(board)

        time.sleep(0.08)


if __name__ == "__main__":
    main()
```